

Learning File 1: The Conceptual Framework

To truly understand BlackLetter, you must first understand the problem it solves and the architecture chosen to solve it.

1. The Core Problem: Context Window Limits You cannot simply paste a 100-page contract into an AI model.

- Issue 1: It is too expensive (you pay per token).
- Issue 2: It is inaccurate (the "Lost in the Middle" phenomenon means AI forgets the middle of long texts).
- Issue 3: It is slow.

2. The Solution: RAG (Retrieval-Augmented Generation) BlackLetter is not just a "chatbot." It is a RAG engine.

Think of it like a student taking an open-book exam.

- The Student = The AI Model (Llama 3).
- The Book = Your PDF Contract.
- The Exam Question = The Checklist Item.

Instead of reading the whole book for every question, the student looks at the index, finds the relevant page, reads ONLY that page, and answers the question.

3. The Application Architecture The app is split into two distinct worlds that talk to each other:

A. The Frontend (React)

- This is the "Client." It runs in your browser.
- It is "dumb" in the sense that it does no AI processing.
- Its only job is to look pretty, take your file, and show you the results.
- It talks to the Backend using HTTP Requests (like a waiter taking an order to the kitchen).

B. The Backend (FastAPI)

- This is the "Server." It runs on your computer (or the cloud).
- It holds the "Brain" (LangChain + Groq).
- It has a "Memory" (ChromaDB).
- It does all the heavy lifting.

4. The Data Flow

5. User uploads PDF -> Frontend sends to Backend.
6. Backend processes PDF -> Saves to Memory (ChromaDB).
7. User clicks Audit -> Frontend sends signal to Backend.
8. Backend retrieves specific pages from Memory -> Sends to AI -> Gets Answer.
9. Backend sends Report -> Frontend displays it.

In the next file, we will explain exactly HOW the "Memory" and "Brain" work mathematically.